Project Assignment 3 Report: Medicaid

Pipeline contract:

- **Project Question**: “Which months had the highest and lowest amount paid out and quantities for each service?”
    - We hope that answering this question will give us a general idea of the impact COVID had on services. We’re also hoping to see if there is a correlation between the amount paid out and the demand of each service.

- **Input File / Expected Schema:** *“medicaid-provider-spending.csv”*
    - Header (comma separated):

BILLING_PROVIDER_NPI_NUM	INTEGER
SERVICING_PROVIDER_NPI_NUM	INTEGER
HCPCS_CODE	VARCHAR
CLAIM_FROM_MONTH	DATE
TOTAL_PATIENTS	INTEGER
TOTAL_CLAIM_LINES	INTEGER
TOTAL_PAID	FLOAT

    - We are expecting to use all fields except for `BILLING_PROVIDER_NPI_NUM`, `SERVICING_PROVIDER_NPI_NUM` and `TOTAL_PATIENTS`

- **Final Output Schema:**

HCPCS_CODE	VARCHAR
HIGHEST_PAID_MONTH	DATE
LOWEST_PAID_MONTH	DATE
HIGHEST_DEMAND_MONTH	DATE
LOWEST_DEMAND_MONTH	DATE

- **Filtering, Missing-Values, Grouping, Ranking and Tie-Handling Rules:**
    - The TOTAL_PAID is the only field with potentially missing data. We found a handful of entries which paid out zero dollars. We’re unsure why this is, but we’re going to filter these out anyways. Some of the HCPCS codes are not valid,

so we will filter those out using regex, since valid HCPCS codes follow a specific format.
    - We expect to group entries with the same HCPCS code using DuckDB
    - The resulting analysis will contain data about each HCPCS code. This means the first column for each entry will contain a unique HCPCS code, so tie-handling is simply not needed.

- **Data-Quality & Parsing risks:**
    - We do not expect there to be any parsing risks, but we do expect risks in data quality. Medicaid fraud is a common issue, and any fluctuations in billing behavior during COVID should be taken with a grain of salt, as there may be a correlation between the frequency of fraud and a public health crisis like COVID.

- **Invariants and Expectations:**
    - One invariant we've been using is that the output file should NOT exceed 8,725. This is the number of possible HCPCS codes we can expect. If the output files exceed this, something has gone wrong, whether it's invalid HCPCS codes being included or the same HCPCS codes with minor differences.
- **Partition ruling:**
    - We will partition by year since it is the most obvious choice

- **Stage Map**
    - `raw-input`
      `-> Remove unnecessary columns, filter out entries with $0 paid out`
      `-> Add the sum of amount paid and claim lines for each HCPCS code with the same month`
      `-> Sort each of the resulting data by month AND by claim lines separately`
      `-> With both of these two resulting files, you can compile a single data file for each HCPCS code which contains the month with most demand, least demand, highest price and lowest price`

Text & Parquet creation process:

It was a lot easier making the text and parquet .sql files over the shell one. Partly because the DuckDB ones run significantly faster, but also because DuckDB is just far more readable. The first thing we do in both is to drop any existing tables, since it could lead to a fatal error otherwise. Then we create a table called hcpcs_sum, which combines all rows with the same HCPCS_CODE and CLAIM_FROM_MONTH by summing the remaining fields together. Then we create a "ranked" table which adds additional fields for where each row should be ranked (this is necessary to ensure tie-breaking is the same as the shell file). Afterwards, we can then

finally create the hcpcs_result table by grouping my hcpcs_codes, and finding the months with the max/min pay out and demand. Once all done we then output the table as a .tsv. The parquet file is identical with just a minor change in how the initial file is read.

Shell, DuckDB & DuckDB-Parquet implementation choices:

The shell script is the slowest implementation out of all them, and the most lengthy one. We first sum patients and amount paid for each unique combination of HCPCS_CODE and CLAIM_MONTH. Then we iterate the outputted file to find the highest/lowest TOTAL_PAID and TOTAL_PATIENTS. The DuckDB scripts do the same thing, but it is more concisely written and a whole lot more performant.

Three-way equivalence evidence:

We tried our absolute best to ensure the data was as close as possible to each other, and we think we did a pretty good job. In our 500 million row data-set, we were able to minimize the difference to a single discrepancy in one of the fields in the 4000 line output. We ensured ties matched between them, arithmetic division and rounding were accurate, but we could not solve this specific discrepancy. We want to chalk it up to some odd unpredictable floating-point precision or rounding phenomena that is just simply not worth trying to resolve.

Nine-run benchmark, text vs Parquet file size interpretation:

We benchmarked all three implementations on the first 1,000,000 records passing our existing filters: TOTAL_PAID > 0 and HCPCS_CODE matching one uppercase letter followed by four digits. This selection is reproducible for an unchanged source file, but is not a random sample. The Parquet input was created from those exact same text records.

Before timing, we used diff to confirm that all three results matched exactly. Each timed result was also checked against the verified reference outside the timed section. All nine runs passed. Bash time measured elapsed time, including process startup, analysis, and result export. Input preparation and Parquet conversion were excluded.

Processing path	**Run 1 (seconds)**	**Run 2 (seconds)**	**Run 3 (seconds)**	**Mean (seconds)**
Shell over text	2.545	2.483	2.482	2.503
DuckDB over text	0.636	0.469	0.501	0.535
DuckDB over Parquet	0.213	0.220	0.221	0.218

The benchmark text file contained 28,633,271 bytes, while Parquet contained 8,987,133 bytes, making Parquet approximately 69% smaller. These sizes refer to the benchmark inputs, not the full source CSV.

DuckDB over Parquet was fastest for this input and analysis. Its average elapsed time was approximately 59% lower than DuckDB over text. Shell and Parquet timings were relatively consistent, while DuckDB-text timings varied more, ranging from 0.469 to 0.636 seconds.

These measurements have limitations. The verification runs warmed the filesystem cache, and the timed runs used a fixed order: shell, DuckDB-text, then DuckDB-Parquet. Shared-machine load, caching, and startup costs may affect the results. We therefore cannot conclude that one tool is universally faster.

The DuckDB queries express grouping and ranking more directly than the shell implementation's awk logic and intermediate files, making them easier to read and maintain. The smaller storage size and faster measured queries support using DuckDB with Parquet in our next project stage, with further testing on larger inputs.

The script is reproducible through scripts/run_benchmark.sh. Measurements are recorded in out/benchmark.tsv, with verification evidence in logs/benchmark_comparison.txt. Agreement on this benchmark does not resolve the separate discrepancy previously reported on the full dataset.

Background evidence:

We ran the entire data-set and provided the output files as requested. Running the entire dataset takes long, as it is bottlenecked by the shell script, which takes the vast majority of the time. Total elapsed time is about 15-20 minutes.

Partition design and three scheduled-run results:

We partitioned by year since it was the most obvious choice. This data ranges from 2018 through to 2024, and we think partitioning the data will be useful and seeing patterns year to year. We hope to use this analysis for our next steps.

Reflection (one failure):

We were really hoping to get the discrepancies between the shell analysis and DuckDB analysis to be zero, but unfortunately we simply could not do this. We spent several hours debugging and improving the discrepancy gap, but closing that last one felt simply impossible. It must be some really odd rounding or floating-point precision quirk that is just far too time consuming to debug.

The role of our analysis in the final project:

We hope this data will help us generally gauge the types of patterns we should expect to see after doing further analysis. We expect our future analysis to be much larger and much more complicated, so it's really important that we do a simpler, more general analysis that is far easier to read, even if it doesn't include the minute details a full analysis would provide.

Next steps:

We're honestly still thinking about what to work on next. This sprint has been pretty exhausting for us, and we need some time to recharge before making any concrete decisions.

Roles:

- **Team Members:**
    - Rupashi – Product Manager (Sprint 3)
    - Marius – Data Engineer (Sprint 3)
    - Dylan – Data Engineer (Sprint 3)
    - Justin - Data Engineer (Sprint 3)
    - Devin – Data Storyteller (Sprint 3) author

AI Disclosure:

AI was used for learning and debugging purposes only; code was hand-written with AI guidance, not AI generated.